

Learn ESP32 for Home Automation

Lesson 8: Building a Smart Light Switch Project

Building a Smart Light Switch Project - Study Notes

Key Concepts

1. **Wi-Fi Connectivity** – Using an ESP-based board (ESP8266/ESP32) to join a local network and communicate with cloud services.
2. **MQTT Protocol** – Lightweight publish/subscribe messaging for sending on/off commands and status updates.
3. **Relay Control** – Driving a mains-rated relay (or solid-state switch) to turn the lamp power on/off safely.
4. **Integration with Mobile & Voice Assistants** – Leveraging Home Assistant, Google Assistant, or Alexa via MQTT discovery.
5. **Safety & Power Isolation** – Proper wiring, opto-isolated relays, and enclosure considerations for mains voltage.

Summary

This lesson walks you through building a **smart light switch** that can be toggled from a smartphone app, a web dashboard, or a voice assistant. The core hardware is an ESP8266/ESP32 development board that connects to your home Wi-Fi network. The board runs firmware that subscribes to an MQTT topic (e.g., `home/livingroom/lamp/set`). When a payload of `"ON"` or `"OFF"` arrives, the firmware drives a relay, which safely switches the 120 V/230 V lamp.

MQTT acts as the glue between the ESP device and the control interfaces. A broker (like Mosquitto) receives commands from Home Assistant, Node-RED, or a simple MQTT client app on your phone and forwards them to the ESP. The ESP also **publishes** its state on a separate topic (e.g., `home/livingroom/lamp/state`), enabling real-time feedback and allowing automations such as “turn off the lamp when no motion is detected”.

The project emphasizes **modular, reusable code**: Wi-Fi connection handling, MQTT reconnection logic, and a clean relay control function. Safety is highlighted—use a properly rated relay, keep high-voltage wiring isolated from the low-voltage ESP side, and house everything in a fire-rated enclosure.

Important Points

- **Hardware Checklist**
 - ESP8266 (NodeMCU, Wemos D1 Mini) **or** ESP32.
 - 5 V or 3.3 V opto-isolated relay module rated for your mains voltage/current.
 - Power supply: 5 V USB adapter or 3.3 V regulator for ESP.
 - Breadboard/prototyping board, jumper wires, enclosure.
- **Software Essentials**

- Arduino IDE / PlatformIO with `ESP8266WiFi` (or `WiFi.h` for ESP32) and `PubSubClient` libraries.
- MQTT broker address, port (usually 1883), and optional credentials.
- OTA (Over The Air) update support for future firmware changes.

- ****MQTT Topics Convention****

...

home/<room>/<device>/set !' payload: ON/OFF

home/<room>/<device>/state !' payload: ON/OFF (published by ESP)

...

Follow Home Assistant discovery format if you want auto entity creation.

- ****Connection Resilience****

- Re attempt Wi Fi connection every few seconds if lost.
- Re connect to MQTT broker on failure; use `client.setCallback()` to handle incoming messages.

- ****Safety Practices****

- ****Never**** power the relay coil directly from the mains; always drive it from the ESP's 3.3 V/5 V rail through the relay module's driver.
- Keep the high voltage side (lamp wiring) physically separated from the low voltage side (ESP).
- Use a ****fuse**** or ****circuit breaker**** rated slightly above the lamp's current draw.
- Verify enclosure is rated for the voltage and temperature of the environment.

- ****Integration Tips****

- Home Assistant: enable MQTT discovery or add a manual `switch` entity.
- Google Assistant / Alexa: expose the MQTT switch via Home Assistant Cloud or use a custom skill that publishes to MQTT.
- Mobile control: use MQTT Dash, IoT MQTT Panel, or a simple web page served by the ESP (optional).

Examples

Example 1 – Minimal Arduino Sketch (ESP8266)

```

```cpp
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
const char* ssid = "YOUR_SSID";
const char* password = "YOUR_PASSWORD";
const char* mqtt_server = "192.168.1.10";
WiFiClient espClient;
PubSubClient client(espClient);
const int RELAY_PIN = D1; // GPIO5 on NodeMCU
const char* setTopic = "home/livingroom/lamp/set";
const char* stateTopic = "home/livingroom/lamp/state";
void setup_wifi() {
 delay(10);
 WiFi.begin(ssid, password);

```

```

while (WiFi.status() != WL_CONNECTED) {
 delay(500);
}
}

void callback(char* topic, byte* payload, unsigned int length) {
 String msg;
 for (unsigned int i = 0; i < length; i++) msg += (char)payload[i];
 if (msg == "ON") digitalWrite(RELAY_PIN, LOW); // assuming active low relay
 if (msg == "OFF") digitalWrite(RELAY_PIN, HIGH);
 client.publish(stateTopic, msg.c_str(), true);
}

void reconnect() {
 while (!client.connected()) {
 if (client.connect("SmartSwitch")) {
 client.subscribe(setTopic);
 } else {
 delay(2000);
 }
 }
}

void setup() {
 pinMode(RELAY_PIN, OUTPUT);
 digitalWrite(RELAY_PIN, HIGH); // start OFF
 Serial.begin(115200);
 setup_wifi();
 client.setServer(mqtt_server, 1883);
 client.setCallback(callback);
}

void loop() {
 if (!client.connected()) reconnect();
 client.loop();
}

```

\*Explanation\*: The sketch connects to Wi-Fi, then to the MQTT broker. It subscribes to `home/livingroom/lamp/set`. When a message arrives, it toggles the relay and immediately publishes the new state to `home/livingroom/lamp/state`. The relay is wired **active low**, so `LOW` turns the lamp on.

---

## Example 2 – Home Assistant Automation

```yaml

configuration.yaml (or via UI)

```
mqtt:
  broker: 192.168.1.10
switch:
  - platform: mqtt
    name: "Living Room Lamp"
    state_topic: "home/livingroom/lamp/state"
    command_topic: "home/livingroom/lamp/set"
    payload_on: "ON"
    payload_off: "OFF"
    optimistic: false
    qos: 1
...
```
```

```yaml

automation.yaml

- alias: "Turn off lamp when no motion for 10 min"

```
trigger:
  - platform: state
    entity_id: binary_sensor.living_room_motion
    to: "off"
    for: "00:10:00"
action:
  - service: switch.turn_off
    target:
      entity_id: switch.living_room_lamp
...

```

Explanation: The MQTT switch entity mirrors the ESP's topics. The automation uses a motion sensor to automatically turn the lamp off after 10 minutes of inactivity, demonstrating how the smart switch can be part of larger home automation logic.

Quick Review

1. **What role does MQTT play in a smart switch project?**
2. **Name two safety measures you must implement when wiring a mains rated relay.**
3. **If the ESP loses its Wi-Fi connection, what should the firmware do?**
4. **How can Home Assistant automatically discover your MQTT switch without manual configuration?**
5. **Why is it recommended to publish the lamp's state back to MQTT?**

Further Reading

- **Deep Dive into MQTT** – QoS levels, retained messages, and Last Will & Testament.
- **ESPHome vs. Arduino Firmware** – Pros/cons for rapid smart home device development.

- ****Secure MQTT**** – TLS/SSL encryption, username password authentication, and token based access.
- ****Power Electronics Safety**** – Understanding IEC/UL standards for DIY mains projects.
- ****Voice Assistant Integration**** – Building custom Alexa Skills or Google Actions that publish MQTT commands.

End of study notes.

